EXPLORATORY DATA ANALYTICS

Module 2

Data Transformation & Wrangling

Merging · Joins · Pivoting · Missing Values · Binning

Mrs. Shilpashree S | Mrs. Vinutha K
Assistant Professors, Dept. of CS & E | 23CSE422

Data Transformation

Data transformation is a set of techniques used to convert data from one format or structure to another, ensuring compatibility and enhancing interoperability.

01 **Data Deduplication**

Identify and remove duplicate data.

02 **Key Restructuring**

- Transform keys with built-in meanings into generic keys.

03 **Data Cleansing**

- Remove outdated, inaccurate, or incomplete information without changing its meaning.

04 **Data Validation**

- Formulate rules or algorithms to validate data against known issues.

08 **Format Revisioning**

- Convert data from one format to another.

06 **Data Derivation**

- Create rules to generate additional information from the source data.

07 **Data Aggregation**

- Searching, extracting, summarize and preserve important information for reporting systems.

08 **Data Integration**

- Convert and merge different data types into a common structure or schema.

09 **Data Filtering**

- Identify and extract relevant information for specific users.

10 **Data Joining**

- Establish relationships between two or more tables.

Merging Database-Style DataFrames

pd.concat() — Vertical Concatenation (axis=0)

Stacking rows from two DataFrames that share the same columns

df1 (Batch A)

	Name	Marks	Grade
0	Alice	82	B
1	Bob	91	A
2	Carol	74	C

df2 (Batch B)

	Name	Marks	Grade
0	David	88	A
1	Eva	65	C

→ axis=0

Result — pd.concat([df1,df2], ignore_index=True)

	Name	Marks	Grade
0	Alice	82	B
1	Bob	91	A
2	Carol	74	C
3	David	88	A
4	Eva	65	C

Python

```
df1 =
pd.DataFrame({'Name':['Alice','Bob','Carol'],
              'Marks':[82,91,74],
              'Grade':['B','A','C']})
df2 = pd.DataFrame({'Name':['David','Eva'],
                    'Marks':[88,65],
                    'Grade':['A','C']})

# Stack vertically – more rows, same columns
result = pd.concat([df1, df2], axis=0,
ignore_index=True)
```

⚠️ Without ignore_index=True the index repeats as 0,1,2,0,1 — causes bugs in .loc[] filtering later.

🌐 Use case: combining monthly CSV files, merging exam batches.

pd.concat() — Horizontal Concatenation (axis=1)

Placing DataFrames side-by-side when they share the same rows

df_SE (SE marks)

	Name	SE_Marks
0	Alice	82
1	Bob	91
2	Carol	74

df_ML (ML marks)

	Name	ML_Marks
0	Alice	88
1	Bob	75
2	Carol	90

→ axis=1

Result — columns placed side-by-side

	Name	SE_Marks	Name	ML_Marks
0	Alice	82	Alice	88
1	Bob	91	Bob	75
2	Carol	74	Carol	90

⚠️ Duplicate 'Name' column — drop it:

	Name	SE_Marks	ML_Marks
0	Alice	82	88
1	Bob	91	75
2	Carol	74	90

Python

```python
# Horizontal concat — axis=1
result = pd.concat([df_SE, df_ML],
axis=1)

# Better: drop the duplicate
'Name' column
result = pd.concat([df_SE,

df_ML[['ML_Marks']]], axis=1)
# Output:
#    Name  SE_Marks  ML_Marks
# 0  Alice     82       88
# 1  Bob       91       75
# 2  Carol     74       90
```

⚠️ axis=1 works only when row indices align perfectly — both DFs must have the same index values in the same order.

🌐 Use case: student performance across subjects; product attributes from multiple tables.

pd.concat() — Horizontal Concatenation (axis=1)

Name can be made as index instead of default index.

df_SE (SE marks)

	Name	SE_Marks
0	Alice	82
1	Bob	91
2	Carol	74

df_ML (ML marks)

	Name	ML_Marks
0	Alice	88
1	Bob	75
2	Carol	90

→ axis=1

Result — columns placed side-by-side

	Name	SE_Marks	Name	ML_Marks
0	Alice	82	Alice	88
1	Bob	91	Bob	75
2	Carol	74	Carol	90

⚠️ Duplicate 'Name' column — drop it:

	Name	SE_Marks	ML_Marks
0	Alice	82	88
1	Bob	91	75
2	Carol	74	90

Python

```python
df_SE = df_SE.set_index('Name')
df_ML = df_ML.set_index('Name')
result = pd.concat([df_SE, df_ML],
axis=1)
print("Concatenated DataFrame:")
display(result)

#Output:
```

```
Concatenated DataFrame:
        SE_Marks  ML_Marks
Name
Alice       82        88
Bob         91        75
Carol       74        90
```

⚠️ axis=1 works only when row indices align perfectly — both DFs must have the same index values in the same order.

🌐 Use case: student performance across subjects; product attributes from multiple tables.

Concatenating along with an axis

pd.concat() Function

pd.concat() is used to concatenate pandas DataFrames along a particular axis (rows or columns).

Syntax:

```
pd.concat([df1, df2], axis=0, ignore_index=False)
```

- axis=0 (default): Concatenates vertically (row-wise).
- axis=1: Concatenates horizontally (column-wise).
- ignore_index=True: Resets the index to a new integer sequence, avoiding duplication of indices.

When to Use pd.concat()?

- When you need to combine data from multiple sources with similar structure.
- When you want to combine data along rows or columns.
- When handling large datasets across multiple files or regions, concatenation helps combine data for further analysis.

Using df.merge

Syntax:

```
pandas.merge(
        left,
        right,
        how='inner',
        on='column_name',
        left_on=None,
        right_on=None,
        left_index=False, right_index=False,
    )
```

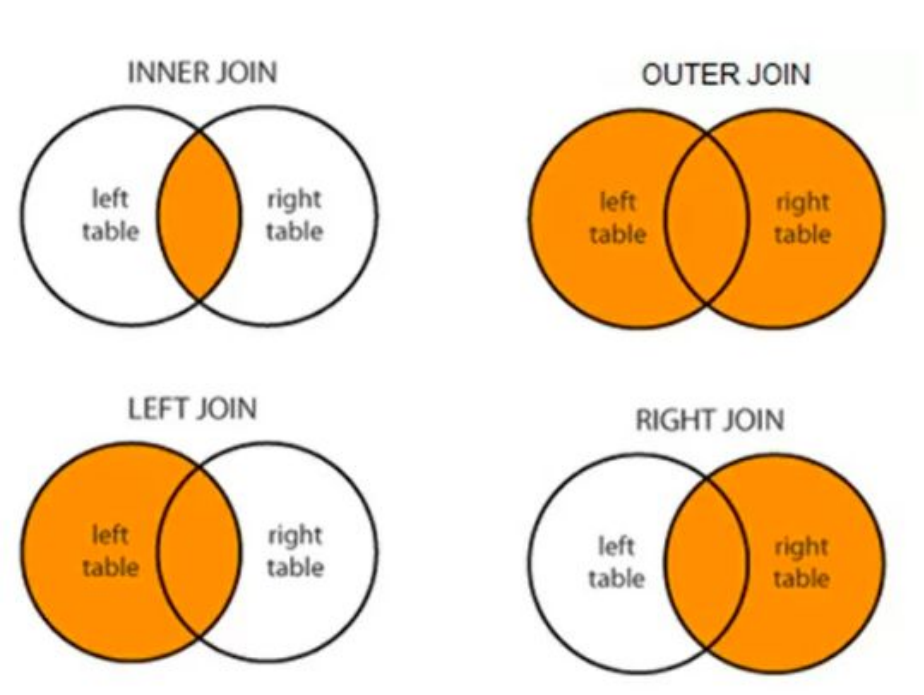
- left, right → DataFrames to be combined
- on → common column(s) used as key
- how → type of join ('inner', 'left', 'right', 'outer')

When keys are different

- left_on → column(s) in left DataFrame used as key
- right_on → column(s) in right DataFrame used as key

When index is used as key

- left_index → use index of left DataFrame as key (True/False)
- right_index → use index of right DataFrame as key (True/False)



df.merge() — INNER JOIN

Returns only rows whose key exists in BOTH DataFrames ≡ SQL INNER JOIN

dfSE (input)

StudentID	SE_Score
S001	82
S002	91
S003	74
S004	88

dfML (input)

StudentID	ML_Score
S001	78
S002	85
S004	90
S005	66

∩

→ INNER

Result (only matched rows)

StudentID	SE_Score	ML_Score
S001	82	78
S002	91	85
S004	88	90

S003 (SE only) → DROPPED

S005 (ML only) → DROPPED

Python

```python
# INNER JOIN — keeps only rows present in BOTH DataFrames
result = dfSE.merge(dfML, how='inner', on='StudentID')

# SQL equivalent:
# SELECT * FROM dfSE INNER JOIN dfML
#   ON dfSE.StudentID = dfML.StudentID

# S003 (SE only) → DROPPED  |  S005 (ML only) → DROPPED
# INNER produces the FEWEST rows of all 4 join types
```

📌 INNER = strictest join. Any row missing from either side is excluded. Row count: INNER ≤ LEFT/RIGHT ≤ OUTER.

🌐 Use case: customers who made both online AND in-store purchases.

df.merge() — LEFT JOIN

All rows from left DataFrame kept; NaN where right has no match ≡ SQL LEFT OUTER JOIN

dfSE (input — PRIMARY)

StudentID	SE_Score
S001	82
S002	91
S003	74
S004	88

dfML (input)

StudentID	ML_Score
S001	78
S002	85
S004	90
S005	66

ALL → LEFT

Result (all 4 SE rows; S003 gets NaN)

StudentID	SE_Score	ML_Score
S001	82	78
S002	91	85
S003	74	NaN
S004	88	90

S003: ML_Score = NaN (row kept)
S005 (ML only) → EXCLUDED

Python

```python
# LEFT JOIN — all left rows kept; NaN where no right match
result = dfSE.merge(dfML, how='left', on='StudentID')

# S003 has no ML record → ML_Score = NaN (row KEPT)
# S005 is ML-only → completely excluded

# Find students with missing ML score:
no_ml = result[result['ML_Score'].isna()]
print(no_ml)    # shows S003
```

📌 LEFT is the most common join — left table is primary, right data supplements it. Missing right data → NaN, not deletion.

🌐 Use case: all registered students + marks for those who appeared (absent students get NaN).

df.merge() — RIGHT JOIN & OUTER JOIN

Completing all four join types

RIGHT JOIN

All ML students; NaN where SE has no match

StudentID	SE_Score	ML_Score
S001	82	78
S002	91	85
S004	88	90
S005	NaN	66

Python

```
result = dfSE.merge(dfML,
         how='right', on='StudentID')
# S003 (SE-only)  → excluded
# S005 (ML-only)  → kept; SE_Score=NaN
```

→ ALL right (dfML)

OUTER JOIN (Full Outer)

ALL students from both; NaN where no match on either side

StudentID	SE_Score	ML_Score
S001	82	78
S002	91	85
S003	74	NaN
S004	88	90
S005	NaN	66

Python

```
result = dfSE.merge(dfML,
         how='outer', on='StudentID')
# S003: ML_Score = NaN
# S005: SE_Score = NaN
# Most rows, most NaN of all joins
```

ALL from both

📌 Row count: INNER ≤ LEFT ≤ OUTER and INNER ≤ RIGHT ≤ OUTER. OUTER = most rows, most NaN.

Merging on Index — left_on & right_index

When the join key lives in the DataFrame index, not a column

left1 (key is a column)

	key	value
0	apple	0
1	ball	1
2	apple	2
3	apple	3
4	ball	4
5	cat	5

right1 (key IS the index)

index	group_val
apple	33.4
ball	5.0

INNER (default) left_on='key', right_index=True

	key	value	group_val
0	apple	0	33.4
1	apple	2	33.4
2	apple	3	33.4
3	ball	1	5.0
4	ball	4	5.0

OUTER — 'cat' has no match

	key	value	group_val
0	apple	0	33.4
1	apple	2	33.4
2	apple	3	33.4
3	ball	1	5.0
4	ball	4	5.0
5	cat	5	NaN

Python

```python
left1 = pd.DataFrame(
    {'key':
['apple','ball','apple','apple','ball','cat'],
     'value': range(6)})

right1 = pd.DataFrame(
    {'group_val': [33.4, 5]},
    index=['apple', 'ball'])
 # <-- key is the INDEX

# Join: left column  <->  right index
inner = pd.merge(left1, right1,
                 left_on='key',
                 right_index=True)
# INNER (default)

outer = pd.merge(left1, right1,
                 left_on='key',
                 right_index=True,
                 how='outer')
# 'cat' row: group_val=NaN
```

🌐 Use case: lookup tables where the index is the key (country code → population).

📌 left_index=True uses left DF's index; right_index=True uses right DF's index. Combine with left_on/right_on as needed.

Reshaping — stack() : Wide → Long Format

Rotating column labels into the row index

dframe1 (WIDE — 3 rows × 5 columns)

	Bergen	Oslo	Trondheim	Stavanger	Kristiansand
Rainfall	0	1	2	3	4
Humidity	5	6	7	8	9
Wind	10	11	12	13	14

→ stack()

stacked (LONG — MultiIndex Series)

Level 0	Level 1	Value
Rainfall	Bergen	0
	Oslo	1
	Trondheim	2
Humidity	Bergen	5
	Oslo	6
	Trondheim	7
Wind	Bergen	10
	Oslo	11
…	…	…

Python

```python
import numpy as np, pandas as pd

data    = np.arange(15).reshape((3, 5))
index   = ['Rainfall', 'Humidity', 'Wind']
cols    = ['Bergen','Oslo','Trondheim','Stavanger','Kristiansand']
dframe1 = pd.DataFrame(data, index=index, columns=cols)
```

```python
# stack(): columns → MultiIndex rows
stacked = dframe1.stack()
print(type(stacked))   # <class 'pandas.core.series.Series'>
print(stacked)   # MultiIndex (15 entries)

# Reverse: unstack() restores original DataFrame exactly
dframe1_restored = stacked.unstack()
```

📌 stack() compresses 15 separate column values into a single Series with 2-level MultiIndex (row label + column name).

🌐 Long format is required for seaborn groupby plots and pandas .groupby() aggregations.

Reshaping — unstack() : Long → Wide + Handling NaN

NaN appears for index combinations that don't exist in the source data

Two Series → concat → MultiIndex

Python

```
series1 = pd.Series([0,111,222,333],
            index=['zeros','ones','twos','threes'])
series2 = pd.Series([444,555,666],
            index=['fours','fives','sixes'])
frame2 = pd.concat([series1, series2],
                   keys=['Number1','Number2'])
```

frame2 (long/stacked form)

Key1	Key2	Value
Number1	zeros	0
Number1	ones	111
Number1	twos	222
Number1	threes	333
Number2	fours	444
Number2	fives	555
Number2	sixes	666

→

unstack()

frame2.unstack() → NaN for missing combos

	fives	fours	ones	sixes	threes	twos	zeros
Number1	NaN	NaN	111	NaN	333	222	0
Number2	555	444	NaN	666	NaN	NaN	NaN

Python

```
# unstack: innermost index level → columns
wide = frame2.unstack()
# NaN fills missing (outer, inner) combinations

# Re-stack to restore long format
long_again = wide.stack(dropna=False)
```

📌 NaN appears wherever a combination like (Number1,fours) never existed in the original data — expected and correct.

Data Cleaning — Detecting & Removing Duplicates

duplicated() · drop_duplicates() · keep parameter

frame3 (original — with duplicates)

	col1	col2
0	Looping	10
1	Looping	10
2	Looping	22
3	Functions	23
4	Functions	23
5	Functions	24
6	Functions	24

duplicated()

Row	isDuplicate
0	False
1	True ✗
2	False
3	False
4	True ✗
5	False
6	True ✗

drop_duplicates() keep='first'

	col1	col2
0	Looping	10
2	Looping	22
3	Functions	23
5	Functions	24

keep='last'

	col1	col2
1	Looping	10
2	Looping	22
4	Functions	23
6	Functions	24

🌐 Use case: deduplicating transaction logs, removing repeated API responses.

Python

```python
frame3 = pd.DataFrame({
    'col1': ['Looping']*3 + ['Functions']*4,
    'col2': [10, 10, 22, 23, 23, 24, 24]})

# Step 1: Detect — True means 'repeat of a previous row'
frame3.duplicated()              # rows 1, 4, 6 → True

# Step 2: Remove — keeps first occurrence by default
frame3.drop_duplicates()

# Keep LAST occurrence instead of first
frame3.drop_duplicates(keep='last')

# Check only specific column for duplicates
frame3.drop_duplicates(subset=['col2'])

# After drop, always clean the index
clean = frame3.drop_duplicates().reset_index(drop=True)
```

⚠️ Always .reset_index(drop=True) after dropping duplicates — otherwise index has gaps (0,2,3,5) that confuse later .iloc[] operations.

Replacing Values — replace()

Fixing sentinel values, typos, and standardising entries

Original (col1 contains sentinel −786)

	col1	col2
0	200.0	0
1	3000.0	1
2	−786.0	2
3	3000.0	3
4	234.0	4
5	444.0	5
6	−786.0	6

→

replace(−786, NaN) → single replacement

	col1	col2
0	200.0	0
1	3000.0	1
2	NaN	2
3	3000.0	3
4	234.0	4
5	444.0	5
6	NaN	6

→

replace({−786:NaN, 0:2}) → multi-replace

	col1	col2
0	200.0	2
1	3000.0	1
2	NaN	2
3	3000.0	3
4	234.0	4
5	444.0	5
6	NaN	6

col2 row 0: 0→2
col1 rows 2,6: −786→NaN

Python

```python
import pandas as pd, numpy as np
df = pd.DataFrame({
    'col1': [200., 3000., -786., 3000., 234., 444., -786.],
    'col2': range(7)})

# Single value replacement
df.replace(to_replace=-786, value=np.nan)

# Multiple values — list form (positional 1-to-1 mapping)
df.replace(to_replace=[-786, 0], value=[np.nan, 2])

# Multiple values — dict form (preferred, explicit)
df.replace({-786: np.nan, 0: 2})

# Column-specific replacement
df['col1'].replace(-786, np.nan, inplace=True)
```

🌐 Use case: survey datasets where −1=refused, 99=unknown, 9999=not applicable.

⚠️ replace() returns a NEW DataFrame by default — assign back: df=df.replace(...) or use inplace=True. Does not modify original.

Missing Data — isnull() / notnull() Detection

Locating NaN values and measuring their extent

dfx (fruit sales — NaN in store4 & store5)

	s1	s2	s3	s4	s5
apple	15	16	17	20.0	NaN
banana	18	19	20	NaN	NaN
kiwi	21	22	23	NaN	NaN
grapes	24	25	26	NaN	NaN
mango	27	28	29	NaN	NaN
waterml	15	16	17	18.0	NaN
oranges	NaN	NaN	NaN	NaN	NaN

dfx.isnull()

	s1	s2	s3	s4	s5
apple	F	F	F	F	T
banana	F	F	F	T	T
kiwi	F	F	F	T	T
grapes	F	F	F	T	T
mango	F	F	F	T	T
waterml	F	F	F	F	T
oranges	T	T	T	T	T

Python

```python
# Count NaN per column
dfx.isnull().sum()
# s1=1  s2=1  s3=1  s4=6  s5=7

# Total NaN in entire DataFrame
dfx.isnull().sum().sum()    # → 16

# Count valid (non-NaN) values per column
dfx.count()
# s1=6  s2=6  s3=6  s4=2  s5=0
```

```python
# Find rows that contain ANY NaN
dfx[dfx.isnull().any(axis=1)]

# Find rows where ALL values are NaN
dfx[dfx.isnull().all(axis=1)]  # →
oranges
```

⚠️ store5 (s5) has 0 valid values — entirely NaN. Drop it with dropna(axis=1,how='all'), do not fill it.

🌐 Use case: data quality audit — identify columns too sparse to be useful before ML pipeline.

Missing Data — dropna() Variants

Before vs After for every parameter combination

Original dfx

	s1	s2	s3	s4	s5
apple	15	16	17	20	NaN
banana	18	19	20	NaN	NaN
kiwi	21	22	23	NaN	NaN
grapes	24	25	26	NaN	NaN
mango	27	28	29	NaN	NaN
waterml	15	16	17	18	NaN
oranges	NaN	NaN	NaN	NaN	NaN

dropna(how='all') → only 'oranges' removed

	s1	s2	s3	s4	s5
apple	15	16	17	20	NaN
banana	18	19	20	NaN	NaN
kiwi	21	22	23	NaN	NaN
grapes	24	25	26	NaN	NaN
mango	27	28	29	NaN	NaN
waterml	15	16	17	18	NaN

dropna(axis=1,how='all') → s5 column removed

	s1	s2	s3	s4
apple	15	16	17	20
banana	18	19	20	NaN
kiwi	21	22	23	NaN
grapes	24	25	26	NaN
mango	27	28	29	NaN
waterml	15	16	17	18
oranges	NaN	NaN	NaN	NaN

s5 dropped (all-NaN column)

Python

```python
# 1. Drop ANY row with ≥1 NaN  →  empty (all rows have NaN)
dfx.dropna()

# 2. Drop rows where EVERY value is NaN  →  removes 'oranges'
dfx.dropna(how='all')

# 3. Drop COLUMNS where all values are NaN  →  removes 's5'
dfx.dropna(axis=1, how='all')

# 4. Drop cols with < 5 valid values  →  also removes 's4'
dfx.dropna(axis=1, thresh=5)

# 5. Keep only rows with ≥4 valid values
dfx.dropna(thresh=4)
```

💡 thresh=N keeps rows/cols with at least N non-NaN values — the most practical parameter for real datasets.

🌐 Use case: drop sensors/columns with >60% missing before ML feature selection.

Missing Data — fillna() : Constant & Statistical Filling

Replace NaN with a fixed value or column statistics — and see the impact on the mean

store4 (original)

Index	store4
apple	20.0
banana	NaN
kiwi	NaN
grapes	NaN
mango	NaN
waterml	18.0
oranges	NaN

→

fillna(0) → NaN → 0.0

Index	store4
apple	20.0
banana	0.0
kiwi	0.0
grapes	0.0
mango	0.0
waterml	18.0
oranges	0.0

→

fillna(mean) → NaN → 19.0

Index	store4
apple	20.0
banana	19.0
kiwi	19.0
grapes	19.0
mango	19.0
waterml	18.0
oranges	19.0

Mean impact — compare strategies

Column	Raw mean	fill=0	fill=mean
store4	19.0	5.43	19.0
store5	NaN	0.0	NaN

fill=0 drags mean from 19.0 down to 5.43 ! fill=mean preserves it.

Python

```python
ser2 = dfx.store4   # apple=20.0, rest NaN, watermelon=18.0

# Fill with constant — fast but distorts statistics
dfx.fillna(0)

# Fill with column mean (only 2 valid: 20+18=38, mean=19)
dfx.fillna(dfx.mean())
```

```python
# Fill per-column with different values
fill_vals = {'store4': dfx.store4.mean(), 'store5': 0}
dfx.fillna(value=fill_vals)

# Compare means before and after
print("Before:", dfx.mean())
print("After 0:", dfx.fillna(0).mean())
print("After mean:", dfx.fillna(dfx.mean()).mean())
```

⚠️ fillna(0) artificially pulls the mean toward zero. fillna(df.mean()) preserves the column's statistical distribution — prefer this for numeric columns.

🌐 Use case: imputing missing temperature readings, filling product ratings before a recommendation model.

Missing Data — ffill · bfill · interpolate()

Smart filling strategies for sequential and time-series data

Original

Index	Val
apple	20.0
banana	NaN
kiwi	NaN
grapes	NaN
mango	NaN
waterml	18.0
oranges	NaN

→

ffill

Index	Val
apple	20.0
banana	20.0
kiwi	20.0
grapes	20.0
mango	20.0
waterml	18.0
oranges	18.0

→

bfill

Index	Val
apple	20.0
banana	18.0
kiwi	18.0
grapes	18.0
mango	18.0
waterml	18.0
oranges	NaN

→

interpolate()

Index	Val
apple	20.0
banana	19.6
kiwi	19.2
grapes	18.8
mango	18.4
waterml	18.0
oranges	NaN

How each method works

Method	Logic	Key detail
ffill	Carries last valid value forward	20→fills banana/kiwi/grapes/mango; 18→fills oranges
bfill	Fills with next valid value backward	18←fills mango/grapes/kiwi/banana; oranges stays NaN
interpolate	Calculates intermediate values linearly	(18−20)/5 steps = −0.4/step → 19.6, 19.2, 18.8, 18.4

Python

```python
ser2 = dfx.store4
ser2.fillna(method='ffill')      # forward fill
ser2.fillna(method='bfill')      # backward fill
ser2.interpolate()               # linear (default)
```

⚠️ oranges stays NaN after bfill and interpolate — no future anchor value exists. Handle separately.

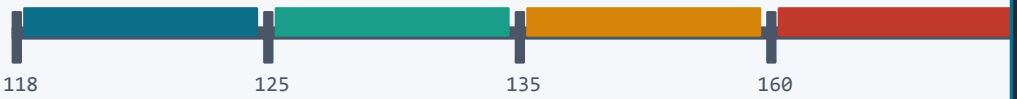
🌐 Use case: stock prices (ffill), IoT sensor gaps (interpolate), survey blanks (bfill).

Discretization — pd.cut() : Custom Bin Edges

Converting continuous values into labelled category intervals

Input: height (continuous)

Idx	Height
0	120
1	122
2	125
3	127
4	121
5	123
6	137
7	131
8	161
9	145
10	141
11	132



Short Average Good Taller

Output: category column

Idx	Height	Category
0	120	Short
1	122	Short
2	125	Short
3	127	Average
4	121	Short
5	123	Short
6	137	Good
7	131	Average
8	161	Taller
9	145	Good
10	141	Good
11	132	Average

Value counts

Category	Count
Short	5
Average	3
Good	3
Taller	1

🌐 Use case: grade bands (90-100=S), BMI categories, age groups, risk tiers.

Python

```python
height =
[120,122,125,127,121,123,137,131,161,145
,141,132]
bins   = [118, 125, 135, 160, 200]   # 4
intervals

# Default: right-closed (a, b] — 125
goes to (118,125]
category = pd.cut(height, bins)

# Add readable labels
bin_names =
['Short','Average','Good','Taller']
category  = pd.cut(height, bins,
labels=bin_names)

pd.value_counts(category)

# Left-closed [a,b) — use right=False
pd.cut(height, [118,126,136,161,200],
right=False)

# Auto equal-width bins (pass integer)
pd.cut(np.random.rand(40), 5,
precision=2)
```

📌 (118,125] means 118 is excluded, 125 is included (right-closed). Use right=False to flip to [118,125).

pd.qcut() vs pd.cut() + Outlier Detection

Equal-frequency binning and filtering extreme values

pd.qcut() — Quantile-based

Edges auto-computed → equal record count per bin

Bin	Range	Count
Q1	(−0.001, 0.261]	500
Q2	(0.261, 0.520]	500
Q3	(0.520, 0.770]	500
Q4	(0.770, 0.999]	500

Python

```python
rnd  = np.random.rand(2000)
# 4 equal-frequency bins (quartiles)
cat3 = pd.qcut(rnd, 4)
pd.value_counts(cat3)  # each ~500
```

```python
# Custom quantile edges
cat4 = pd.qcut(rnd, [0, 0.3, 0.5,
0.7, 1.0])
```

	cut()	qcut()
Edges	User-defined	Auto (quantiles)
Bin width	Equal	Varies
Records/bin	Varies	~Equal

Outlier Detection — np.abs() threshold

Flag rows where |TotalPrice| exceeds a threshold

Idx	UnitPrice	Qty	TotalPrice
0	12.50	100	1,250
1	45.00	150	6,750
2	8.00	50	400
3	1200.00	6000	7,200,000
4	3.50	200	700

Python

```python
df['TotalPrice'] = df['UnitPrice']*df['Quantity']
total = df['TotalPrice']
# Threshold method
outliers = total[np.abs(total) > 3_000_000]
# IQR method (statistically grounded)
Q1, Q3 = total.quantile(0.25), total.quantile(0.75)
IQR = Q3 - Q1
df[(total < Q1-1.5*IQR)|(total > Q3+1.5*IQR)]
```

📌 qcut() = equal record counts per bin. cut() = equal-width ranges. IQR method is statistically grounded — same rule as box plots.

get_dummies() — One-Hot Encoding Categorical Variables

ML models need numbers, not text — convert each category to a binary 0/1 column

Original DataFrame

	gender	votes
0	female	6
1	female	7
2	male	8
3	unknown	9
4	male	10
5	female	11

→

get_dummies(prefix='gender') → 3 binary columns

	g_female	g_male	g_unknown
0	1	0	0
1	1	0	0
2	0	1	0
3	0	0	1
4	0	1	0
5	1	0	0

→

drop_first=True → 2 columns (avoids dummy trap)

	g_male	g_unknown
0	0	0
1	0	0
2	1	0
3	0	1
4	1	0
5	0	0

g_female dropped:
if g_male=0 AND g_unknown=0
→ must be female (redundant)

Python

```python
df =
pd.DataFrame({'gender':['female','female','male','unknown','male',
'female'],
                    'votes': range(6,12)})

# Basic encoding
dummies = pd.get_dummies(df['gender'])

# With prefix — essential when encoding multiple columns
dummies = pd.get_dummies(df['gender'], prefix='gender')
```

```python
# With prefix — essential when encoding multiple columns
dummies = pd.get_dummies(df['gender'], prefix='gender')

# Avoid the dummy trap — drop one column per categorical feature
dummies = pd.get_dummies(df['gender'], prefix='gender',
drop_first=True)

# Attach back to original DataFrame
df_enc = pd.concat([df.drop('gender', axis=1), dummies], axis=1)
```

⚠️ THE DUMMY TRAP: k categories need only k−1 dummy columns. The k-th is always predictable from the rest → causes multicollinearity in regression models.

🌐 Use case: one-hot encoding for logistic regression, SVM, neural networks — same as sklearn's OneHotEncoder.

Renaming Indexes — map() and rename()

Transforming row and column labels programmatically

dframe1 (original)

	Bergen	Oslo	Trondheim	Stavanger	Kristiansand
Rainfall	0	1	2	3	4
Humidity	5	6	7	8	9
Wind	10	11	12	13	14

After index.map(str.upper)

	Bergen	Oslo	Trondheim	Stavanger	Kristiansand
RAINFALL	0	1	2	3	4
HUMIDITY	5	6	7	8	9
WIND	10	11	12	13	14

After rename(index=str.title, columns=str.upper)

	BERGEN	OSLO	TRONDHEIM	STAVANGER	KRISTIANSAND
Rainfall	0	1	2	3	4
Humidity	5	6	7	8	9
Wind	10	11	12	13	14

Python

```python
# map() — uniform transform on all index labels
dframe1.index = dframe1.index.map(str.upper)
# Rainfall→RAINFALL, Humidity→HUMIDITY, Wind→WIND

# map() with lambda
dframe1.index = dframe1.index.map(lambda x: x[:3])
# Rainfall→Rai, Humidity→Hum

# rename() — transforms index AND columns simultaneously
dframe1.rename(index=str.title, columns=str.upper)

# rename() with specific dict mapping (selective)
dframe1.rename(index={'Rainfall': 'Precipitation'},
               columns={'Bergen': 'BER', 'Oslo': 'OSL'})

# When to use:
# map()    → uniform transform (every label through same
function)
# rename() → mixed/selective (index and columns
independently)
```

🌐 Use case: standardising column names (lowercase, strip spaces) before merging two DataFrames.

📌 map() modifies index in-place (reassign needed). rename() returns new DF — assign back or use inplace=True.

Permutation & Random Sampling

Shuffling rows and selecting subsets — essential for ML train/test splits

Permutation — shuffle ALL rows

Original

Idx	A	B
0	0	1
1	8	9
2	16	17
3	24	25

→

After permutation

Idx	A	B
1	8	9
3	24	25
0	0	1
2	16	17

Python

```
dat = np.arange(80).reshape(10, 8)
df  = pd.DataFrame(dat)
# Permute all row indices randomly
sampler  = np.random.permutation(10)
shuffled = df.take(sampler)
# Modern: df.sample(frac=1,random_state=42)
#             .reset_index(drop=True)
```

80% train: df.sample(frac=0.8, random_state=42)

Sampling — select a subset

WITHOUT replacement (no repeats)

Python

```
perm = np.random.permutation(len(df))
df.take(perm[:3])              # 3 unique rows
# Modern: df.sample(n=3, random_state=42)

# 80/20 train-test split
train = df.sample(frac=0.8, random_state=42)
test  = df.drop(train.index)
```

WITH replacement (repeats allowed)

Python

```
sack    = np.array([4, 8, -2, 7, 5])
sampler = np.random.randint(0,len(sack),size=10)
draw    = sack.take(sampler)
# [7,7,4,5,4,4,8,-2,8,5] — repeats occur
# Modern: pd.Series(sack).sample(n=10,replace=True)
```

💡 Always set random_state=42 for reproducibility — same split every run. Essential for ML experiments.

Module 2 — Quick Reference Cheatsheet

Operation	Key Function / Syntax	Critical Note
Vertical concat	pd.concat([df1,df2], axis=0, ignore_index=True)	ignore_index resets to 0,1,2,... avoids duplicate indices
Horizontal concat	pd.concat([df1,df2], axis=1)	Row indices must align; drop duplicate columns afterward
Inner join	df1.merge(df2, how='inner', on='key')	Only matching rows — fewest rows, no NaN from join
Left join	df1.merge(df2, how='left', on='key')	All left rows kept; NaN where right has no match
Right join	df1.merge(df2, how='right', on='key')	All right rows kept; NaN where left has no match
Outer join	df1.merge(df2, how='outer', on='key')	All rows from both; most rows, most NaN
Merge on index	pd.merge(l, r, left_on='col', right_index=True)	right_index=True uses the right DF's index as join key
Wide → Long	df.stack()	Columns → MultiIndex rows; result is a Series
Long → Wide	series.unstack()	Index rows → columns; NaN for missing combinations
Detect duplicates	df.duplicated()	True = repeat of an earlier row
Remove duplicates	df.drop_duplicates(keep='first')	keep='last' or subset=['col'] for column-specific check
Replace values	df.replace({-786: np.nan})	Returns new DF — assign back or use inplace=True

Module 2 — Quick Reference Cheatsheet

Operation	Key Function / Syntax	Critical Note
Detect NaN	df.isnull().sum()	Per column; .sum().sum() for total NaN count
Drop NaN	df.dropna(how='all', thresh=N, axis=1)	how='all' = only all-NaN rows; thresh=N = flexible cutoff
Fill NaN constant	df.fillna(0) or df.fillna(df.mean())	mean fill preserves column distribution; 0 distorts mean
Fill NaN sequence	df.fillna(method='ffill') or 'bfill'	ffill = carry forward; bfill = carry backward
Interpolate NaN	df.interpolate(method='linear')	Estimates between two known anchor points
Custom bins	pd.cut(data, bins, labels=[...])	right=False for left-closed [a,b) intervals
Equal-freq bins	pd.qcut(data, 4)	Quartiles; each bin gets ~equal record count
Outlier detection	df[np.abs(col) > threshold]	IQR: flag below Q1−1.5×IQR or above Q3+1.5×IQR
Dummy encoding	pd.get_dummies(df['col'], prefix='p', drop_first=True)	drop_first=True avoids dummy trap / multicollinearity
Shuffle rows	df.sample(frac=1, random_state=42).reset_index(drop=True)	Train/test: df.sample(frac=0.8, random_state=42)